Reinforcement Learning, Cart-Pole

EN.530.641 Statistical Learning for Engineers

Weixuan Li

1 Problem Statement

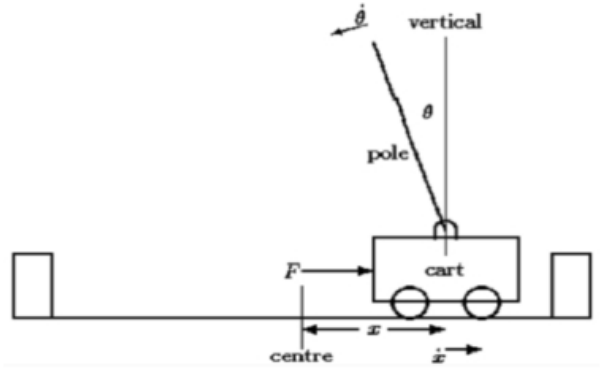

Figure 1: A schematic of the cart-pole system.

In this task, you will apply reinforcement learning techniques to a 2-D inverted pendulum (cart-pole system). See Fig. 1 for a schematic configuration. Specifically, you will apply **Dynamic programming (DP):**

Try Policy Iteration and Value Iteration to compare. Note that you do not use a model defined in `CartPole-v0`, i.e., any `env.step()` function. You have to implement the model (and the algorithms) in your Python codes to perform the task.

2 The model for DP

Dynamic programming (DP) requires a model. For simplicity, let us consider the cart as a point (or particle) with mass M. Also, the pole (pendulum) has the mass m with the half-pole length l. The model is written as the following nonlinear equations [**?**]:

$$\ddot{\theta} = \frac{g \sin\theta + \cos\theta \left(\frac{-F - ml\dot{\theta}^2 \sin\theta + \mu_c \mathrm{sgn}(\dot{x})}{M+m} \right)}{l \left(\frac{4}{3} - \frac{m\cos^2\theta}{M+m} \right)} - \frac{\mu_p \dot{\theta}}{ml}, \tag{1}$$

$$\ddot{x} = \frac{F + ml\left(\dot{\theta}^2 \sin\theta - \ddot{\theta}\cos\theta\right) - \mu_c \text{sgn}(\dot{x})}{M + m}. \tag{2}$$

Here, F is the force applied to the cart. The state of the system is defined as $s = \left[\theta, \dot{\theta}, x, \dot{x}\right]^T$ (or in vector form). The system equations can be numerically solved by the simple Euler's method as:

$$\begin{aligned} \dot{x}(t+1) &= \dot{x}(t) + \Delta t \ddot{x}(t), \\ x(t+1) &= x(t) + \Delta t \dot{x}(t), \\ \dot{\theta}(t+1) &= \dot{\theta}(t) + \Delta t \ddot{\theta}(t), \\ \theta(t+1) &= \theta(t) + \Delta t \dot{\theta}(t), \end{aligned} \tag{3}$$

where t denotes the index of the discretized time. Equations (1) and (2) can be used to compute $p(s' \mid s, a)$ by using the Boxes system below. Note that you use tabular solution methods in this task.

3 Methodology and Results

Considering the cart-pole system with four variables $s = (\theta, x, \dot{\theta}, \dot{x})$, we implement the Boxes system to discretize the state set. The discretized spaces of the system are:

θ	$(-12°, -6°)$	$(-6°, -1°)$	$(-1°, 0)$	$(0, 1°)$	$(1°, 6°)$	$(6°, 12°)$
x	$(-2.4\,\text{m}, -0.8\,\text{m})$	$(-0.8\,\text{m}, 0.8\,\text{m})$	$(0.8\,\text{m}, 2.4\,\text{m})$	/	/	/
$\dot{\theta}$	$(-\infty, -50°/\text{s})$	$(-50°/\text{s}, 50°/\text{s})$	$(50°/\text{s}, +\infty)$	/	/	/
$\dot{x}$	$(-\infty, -0.5\,\text{m/s})$	$(-0.5\,\text{m/s}, 0.5\,\text{m/s})$	$(0.5\,\text{m/s}, +\infty)$	/	/	/

Table 1: Discretized spaces for the cart-pole system.

This generates $6 \times 3 \times 3 \times 3 = 162$ regions through all combinations of the intervals. The actions are $a = \{-10\,\text{N}, 10\,\text{N}\}$. Then we implement the Dynamic Programming to solve the problem.

Using corresponding nonlinear equations as the dynamic model and Euler's method, the transition probability $p(s' \mid s, a)$ can be computed. The details of how we compute it are explained here. First, we select a large *amount* of points (10000 in our code) randomly with uniform probability in each region s. Under each action a, we calculate the next state s' by the dynamic model for each selected point. Then the number of points reaching the next state $N(s' \mid s, a)$ divided by the number of total points N represents the transition probability:

$$p(s' \mid s, a) = \frac{N(s' \mid s, a)}{N}.$$

After calculating the transition probability, we implement the Policy Iteration and Value Iteration to obtain the value function and optimal policy for this problem. The results of Policy Iteration are shown in Fig. 2 and the results of Value Iteration are shown in Fig. 3.

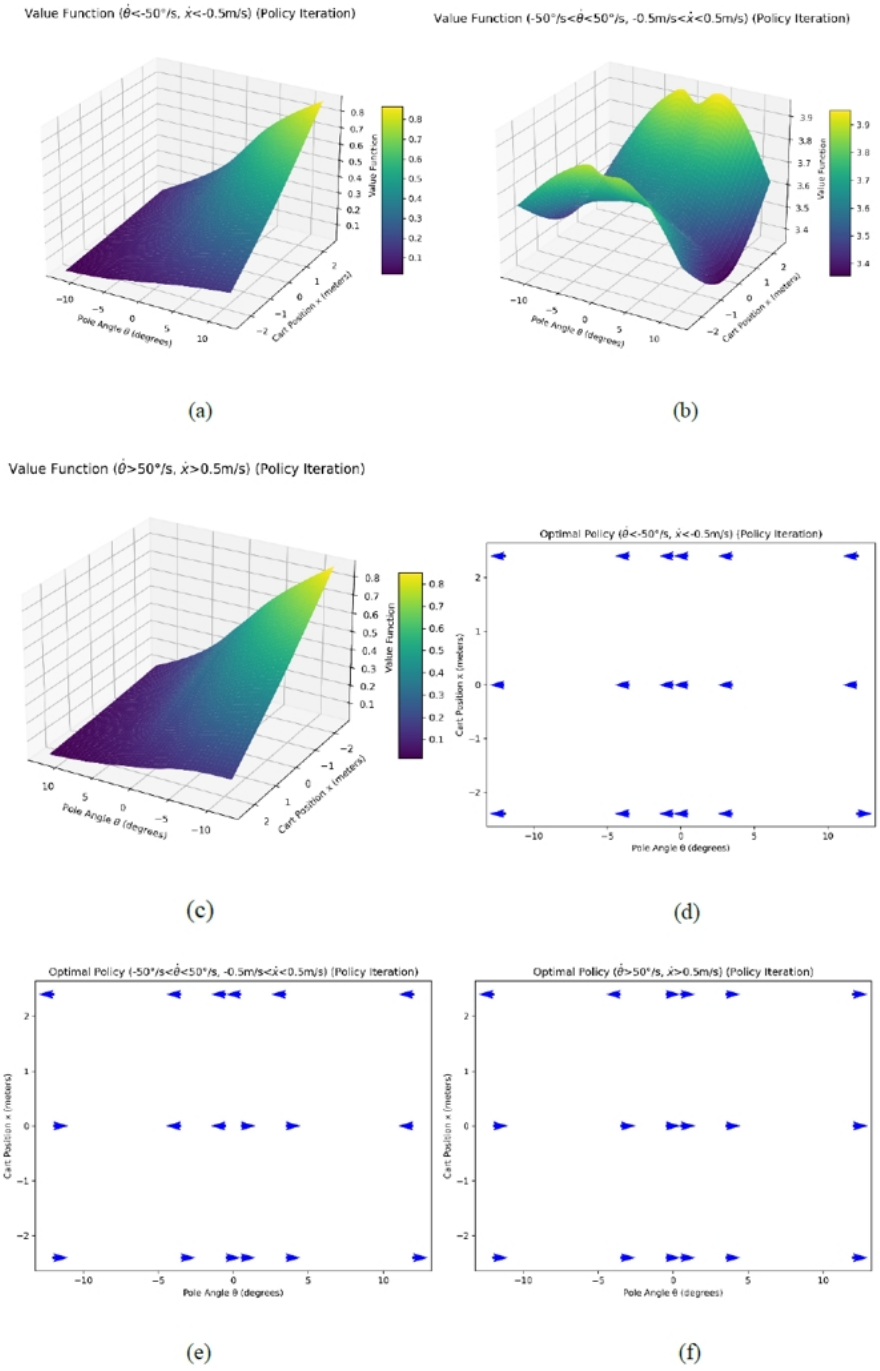

Figure 2: Value Function and Optimal Policy of Policy Iteration.

The results obtained from Policy Iteration and Value Iteration are quite similar, despite some minor differences. The optimal policy aligns well with physical intuitions. As for the value function, when the velocity and angular velocity are towards left, i.e.,

$$\dot{\theta} < -50^{\circ}/\mathrm{s},\ \dot{x} < -0.5\,\mathrm{m/s},$$

the value function is larger when the angle and position are further to the right, since these conditions are safer in this case. On the contrary, when the velocity and angular

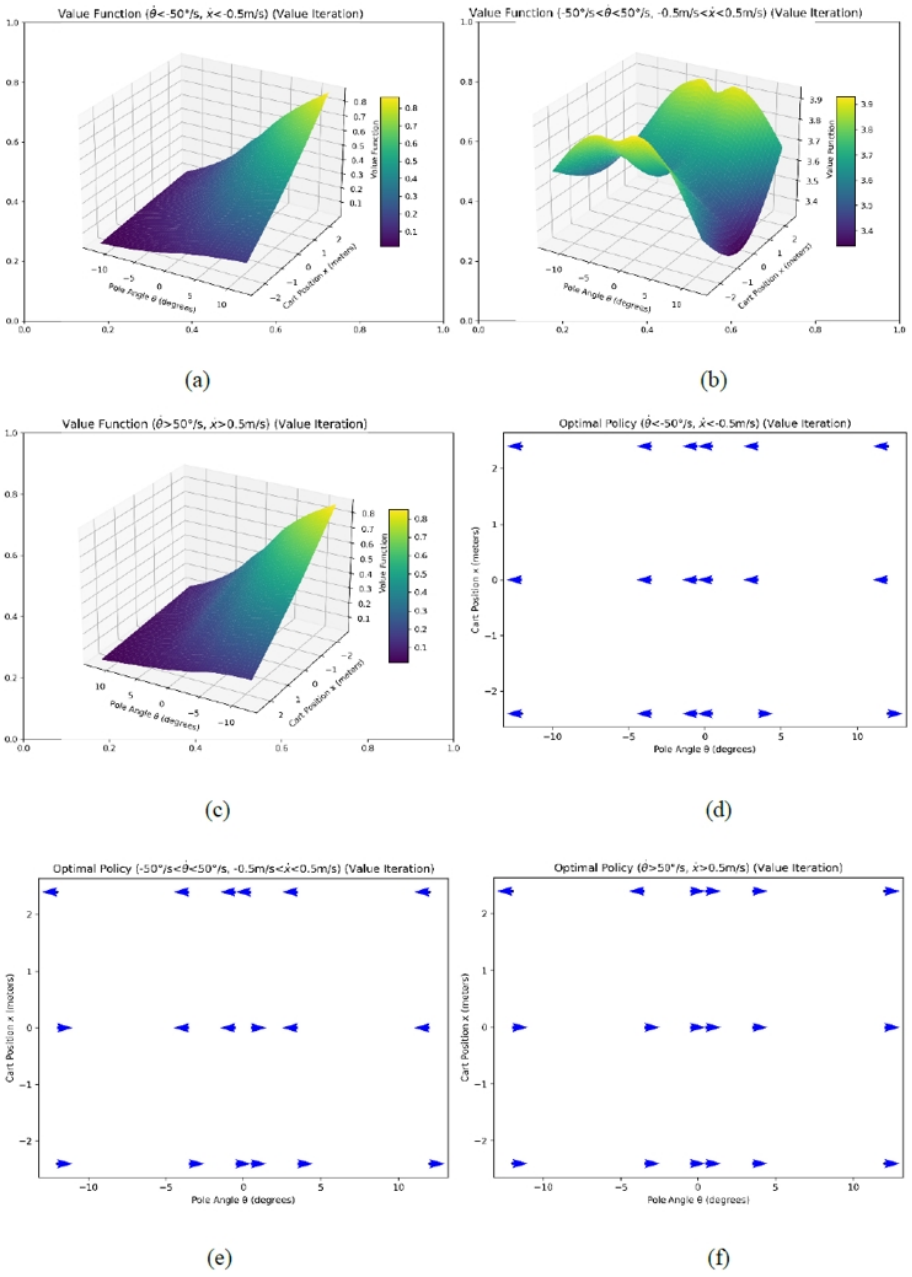

Figure 3: Value Function and Optimal Policy of Value Iteration.

velocity are towards right, i.e.,

$$\dot{\theta} > 50^\circ/\text{s},\ \dot{x} > 0.5\,\text{m/s},$$

the value function is larger when the angle and position are further to the left, since these conditions are safer in this case. When the velocity and angular velocity are mediate, i.e.,

$$-50^\circ/\text{s} < \dot{\theta} < 50^\circ/\text{s},\ -0.5\,\text{m/s} < \dot{x} < 0.5\,\text{m/s},$$

the value function at the center is small. That is because in this case, the best policy at the center is not to apply too much force on the cart. Therefore, neither applying force to the left nor applying force to the right would have a high value.